DevOps for React Developers

From Code to Production

(WORKSHOP)

Author: Kristiyan Velkov

www.kristiyanvelkov.com

Agenda

1. About me

2. What is Docker and Docker Architecture ?

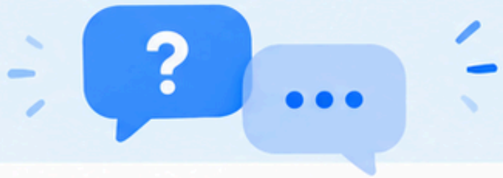
3. Why Docker Matters for React Developers?

4. Quick Break - 15 min.

5. Dockerize a React.js Sample Application.

6. GitHub Actions CI/CD: Build & Push to Docker Hub

7. Q&A



ABOUT ME

Kristiyan Velkov

SPEAKER · DOCKER CAPTAIN · CURSOR AMBASSADOR
BOOK AUTHOR & TECH BLOGGER

Build
Learn
Share
Grow

Most popular Bulgarian speaker
at international tech conferences

12+ years
in modern web development, specializing in
React.js, Next.js, Angular, and Vue.js

7+ years
of hands-on DevOps expertise, including
CI/CD, Docker, and cloud infrastructure

Published author of multiple IT books
focused on front-end development
and best practices

Followed by over 50,000 tech professionals
across Medium, LinkedIn, and other platforms

100+ industry-recognized IT certifications
spanning front-end, DevOps, cloud, and security domains

- Speaker
- Docker Captain
- Cursor Ambassador
- Book Author & Tech Blogger

www.kristiyanvelkov.com

Same Developer
Bigger Possibilities.

RESOURCES

DOCKER GUIDES I AUTHORED

Share Knowledge Build Better Developers

docker

Practical, up-to-date guides to help developers build, ship and scale with Docker.

DOCKER OFFICIAL NODE.JS GUIDE

Build, run and deploy Node.js applications with Docker.

DOCKER OFFICIAL REACT.JS GUIDE

Containerize and deploy modern React applications.

DOCKER OFFICIAL ANGULAR GUIDE

Build and deploy Angular applications with Docker.

DOCKER OFFICIAL VUE.JS GUIDE

Containerize and deploy Vue.js applications.

AZURE PIPELINES WITH DOCKER

Build, test and deploy with Azure DevOps and Docker.

DOCKER OFFICIAL NEXT.JS GUIDE

Build and deploy Next.js applications with Docker.

DOCKER OFFICIAL TANSTACK START GUIDE

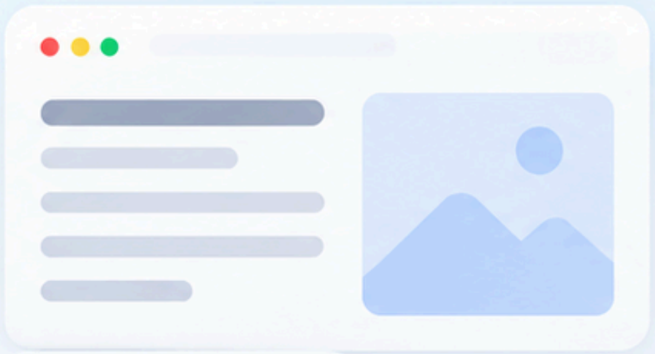
Get started with TanStack using Docker.

MY PUBLICATIONS

250,000+ People Reached Annually

docker

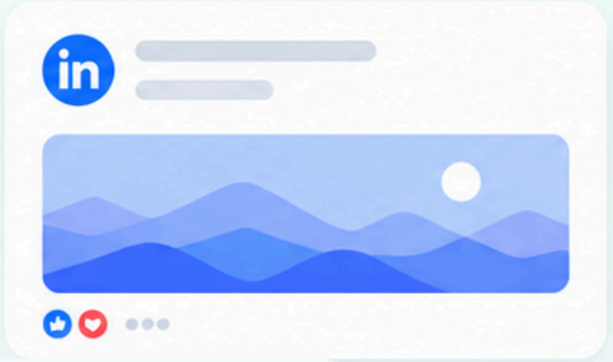
Medium



7,000+
FOLLOWERS

TOPICS
React.js, Angular, Next.js, TypeScript, Docker, Productivity

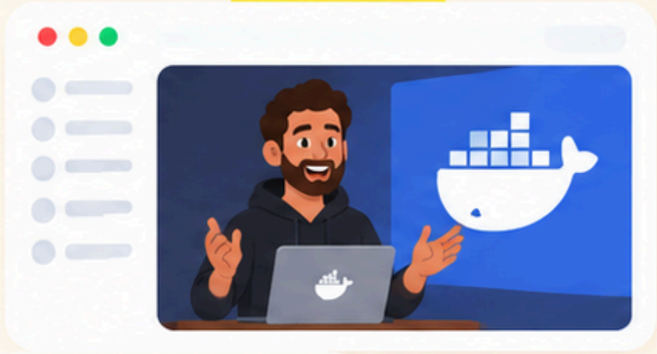
LinkedIn



30,000+
FOLLOWERS

TOPICS
React.js, Angular, Next.js, TypeScript, Docker, Productivity

FE
Front-end WORLD



3,500+
FOLLOWERS

TOPICS
React.js, Angular, Next.js, TypeScript, Docker, DevOps, Productivity, People Management

www.kristiyanvelkov.com

The History of Docker

From a simple idea to the standard for containers

2008

Solomon Hykes and his team founded **dotCloud**.

2010–2012

dotCloud built an internal tool to make containers easier to use – **Docker**.

March 2013

Docker was released as **open-source**.

2013–2014

The **community exploded**; developers finally had a practical, standardized way to package and run apps.

2014

dotCloud rebranded to **Docker, Inc**.

Today

Docker remains **the standard** for building, shipping, and running applications — the tool every developer expects to know.

Solomon Hykes
Co-founder of Docker

What is Docker and Docker Architecture ?

Docker packages your application and everything it needs into portable, isolated containers.

Registry / Docker Hub
Stores and distributes images

Push images

Pull images

Developer

Docker Client (CLI)

```
$ docker build .
$ docker run
$ docker compose up
```

Docker Engine

- Builds images
- Runs containers
- Manages resources

Images

Immutable templates

Containers

Running instances of images

Docker Client (CLI)
Command-line tools to build, run and manage containers.

Docker Engine
Builds images, runs containers and manages the Docker runtime.

Images
Read-only templates that contain your application and dependencies.

Containers
Lightweight, isolated and running instances of images.

Registry (Docker Hub)
Stores and distributes Docker images.

What is Docker Engine ?

Docker is built around two main parts: the Docker Client and the Docker Engine (the server). The Docker CLI is the tool you use to interact with Docker. Think of it as the remote control, and the Docker Engine as the machine that actually performs all the work.

Docker Client (CLI)

```
$ docker build .
$ docker run
$ docker compose up
```

Terminal / Your machine

Send commands →

← Get results

Docker Engine

- Builds images
- Runs containers
- Manages resources

Images

Immutable templates with your application

Containers

Running instances of images

Push images ↓ ↑ Pull images

Registry / Docker Hub

Stores and distributes Docker images

Docker Workflow

From source code to a running container

docker

1 Image Runtime Pipeline

Build the Docker image

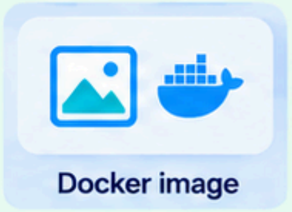
Your application code

Dockerfile

```
$ docker build
  -t my-app .
```

2 Container Runtime Pipeline

Run the Docker image



Docker image

(from Step 1)

```
$ docker run
  my-app
```

3 Docker Engine

Receives requests and manages the process

Engine API
Receives build and run requests

dockerd (daemon)
Executes build and run pipeline

4 Image Build Pipeline

Uses buildkit to create the image

buildx
(CLI plugin)
Parses build options (cache, multi-stage build)

BuildKit
(image builder)
Processes the Dockerfile and builds image layers

Docker image
my-app:latest

5 Container Run Pipeline

Uses containerd to run the container

containerd
Container lifecycle management (start, stop, delete)

shim
Keeping container open

Running container
my-app

What is a Dockerfile ?

A Dockerfile is a simple text file that defines how to build a Docker image for your application.
It contains step-by-step instructions—like which Node version to use, how to install dependencies, how to build your React app, and how to prepare the final production image.

Dockerfile

```
1  FROM node:20-alpine
2  WORKDIR /app
3  COPY package*.json ../
4  RUN npm install
5  COPY . .
6  RUN npm run build
7  EXPOSE 3000
8  CMD ["npm", "start"]
```

Dockerfile

A text file with step-by-step instructions to build your image.

Docker Image

A read-only template that contains your app, runtime, dependencies and configuration.

Running container(s)

A lightweight, isolated runtime environment created from the image that runs your application.

What is a Docker Image?

A Dockerfile is a simple text file that defines how to build a Docker image for your application.
It contains step-by-step instructions—like which Node version to use, how to install dependencies, how to build your React app, and how to prepare the final production image.

Dockerfile

```
1  FROM node:20-alpine
2  WORKDIR /app
3  COPY package*.json ./
4  RUN npm install
5  COPY . .
6  RUN npm run build
7  EXPOSE 3000
9  CMD ["npm", "start"]
```

App code

Dependencies

Node.js

Alpine Linux

Isolated runtime

Own filesystem

All dependencies included

Dockerfile

A text file with step-by-step instructions to build your app.

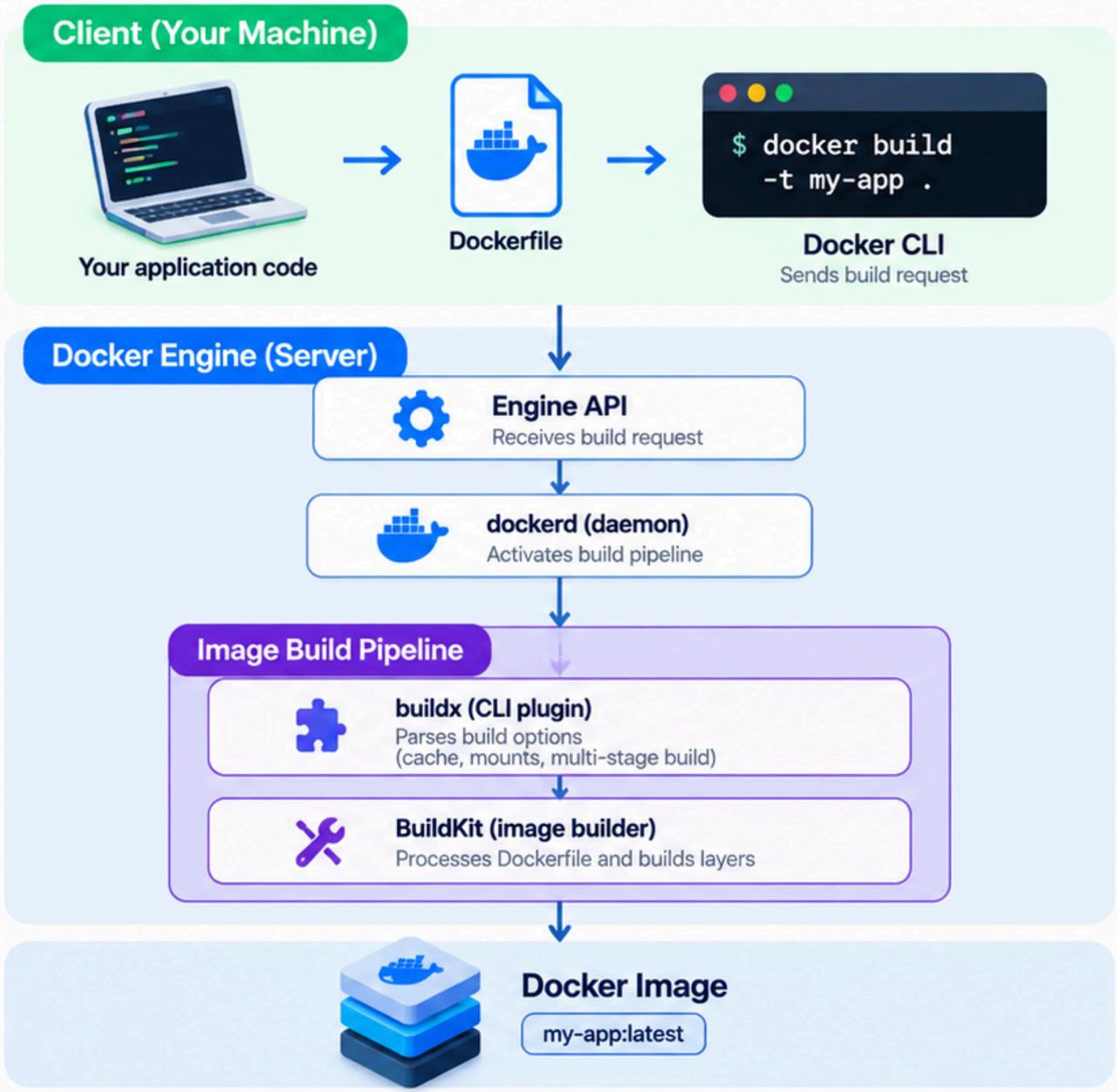
Docker Image

A read-only template that contains your app, runtime, dependencies and configuration.

Running container(s)

A lightweight, isolated runtime environment created from the image that runs your application.

Docker Image Build Pipeline



1. **Your application code**
   You write your application and a Dockerfile that defines how to build it.

2. **Docker CLI**
   You run `docker build -t my-app .` which sends the build request to the Docker Engine.

3. **Engine API**
   The Docker Engine API receives the request.

4. **dockerd (daemon)**
   The Docker daemon activates the image build pipeline.

5. **buildx (CLI plugin)**
   Parses build options (cache, mounts, multi-stage build).

6. **BuildKit (image builder)**
   Processes the Dockerfile and builds the image layers.

7. **Docker Image**
   The result is a Docker image (e.g. `my-app:latest`) that you can run, share, or deploy.

What is a Docker Container?

A Docker container is a lightweight, isolated runtime environment created from a Docker image. It runs your application with everything it needs—its own filesystem, libraries, dependencies, and configuration—fully separated from the host machine.

1 Dockerfile

Defines the application environment, dependencies and configuration.

```
FROM node:20
WORKDIR /app
COPY . .
RUN npm install
CMD ["npm", "start"]
```

Dockerfile

2 Docker Image

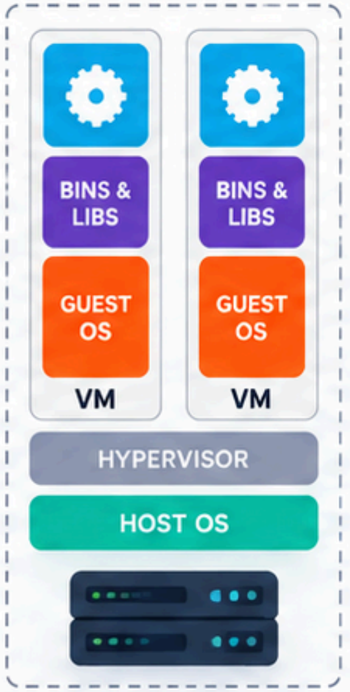
A read-only template that contains everything your app needs.

Docker Image

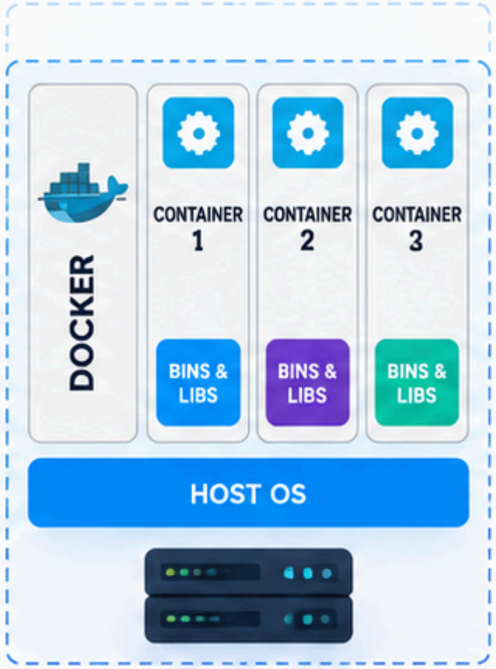
3 Running Container

A lightweight, isolated runtime environment created from the image.

Running Container

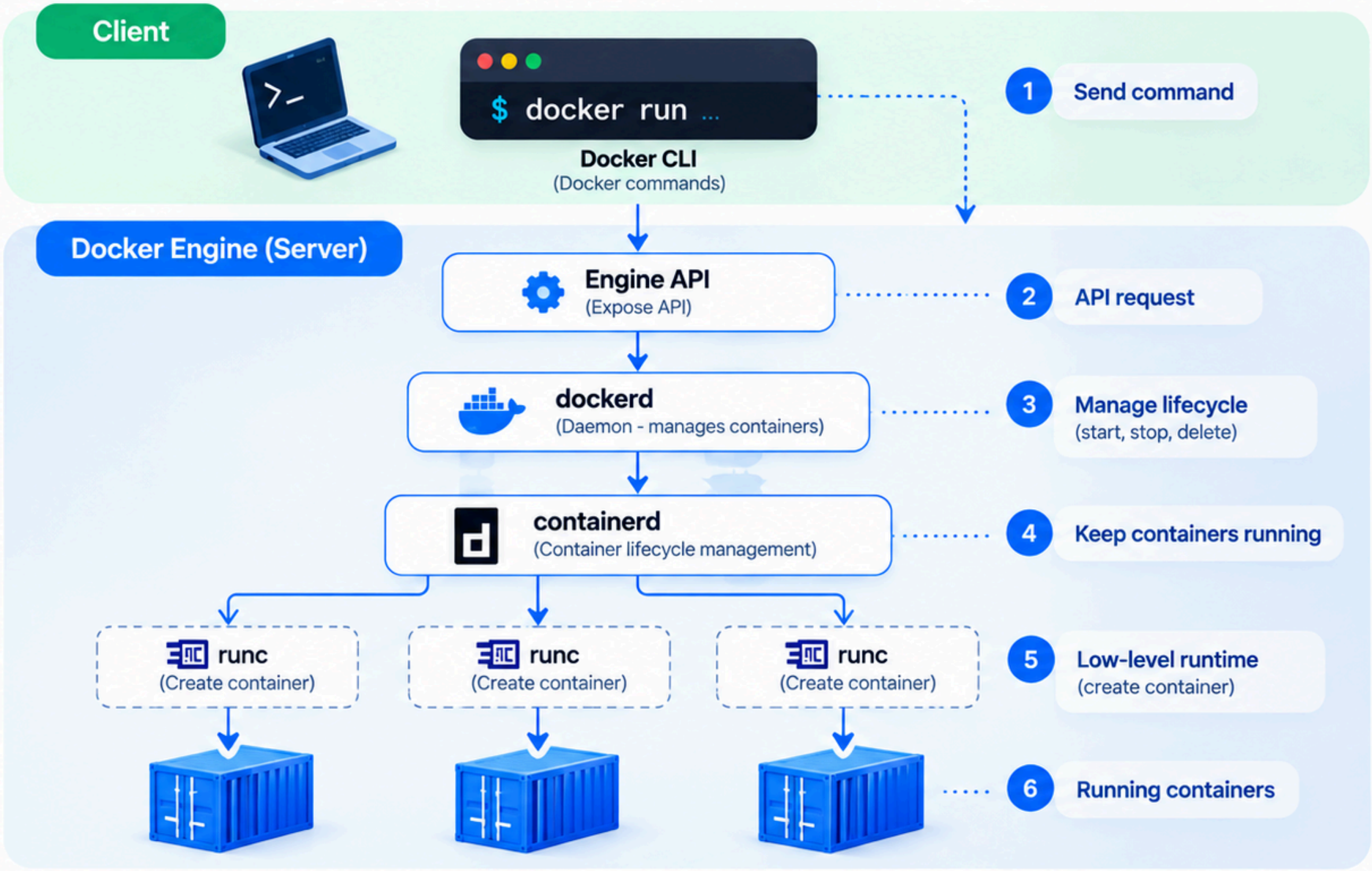


Server with Virtual Machines



Server with Docker Containers

Container Runtime Pipeline



Why Docker Matters for React Developers ?

docker

Fewer conflicts

Every project runs in its own clean, isolated environment. No “works on my machine” excuses.

Reliable deployments

You ship the exact same build you tested locally, every single time.

Rock-solid CI/CD

Consistent pipelines with predictable builds and zero dependency drift.

Real DevOps skills

You instantly stand out as a modern front-end engineer who understands how apps run in the real world.

Why Docker Matters for React Developers ?

docker

Fewer conflicts

Every project runs in its own clean, isolated environment. No “works on my machine” excuses.

Reliable deployments

You ship the exact same build you tested locally, every single time.

Rock-solid CI/CD

Consistent pipelines with predictable builds and zero dependency drift.

Real DevOps skills

You instantly stand out as a modern front-end engineer who understands how apps run in the real world.

REMEMBER!!!

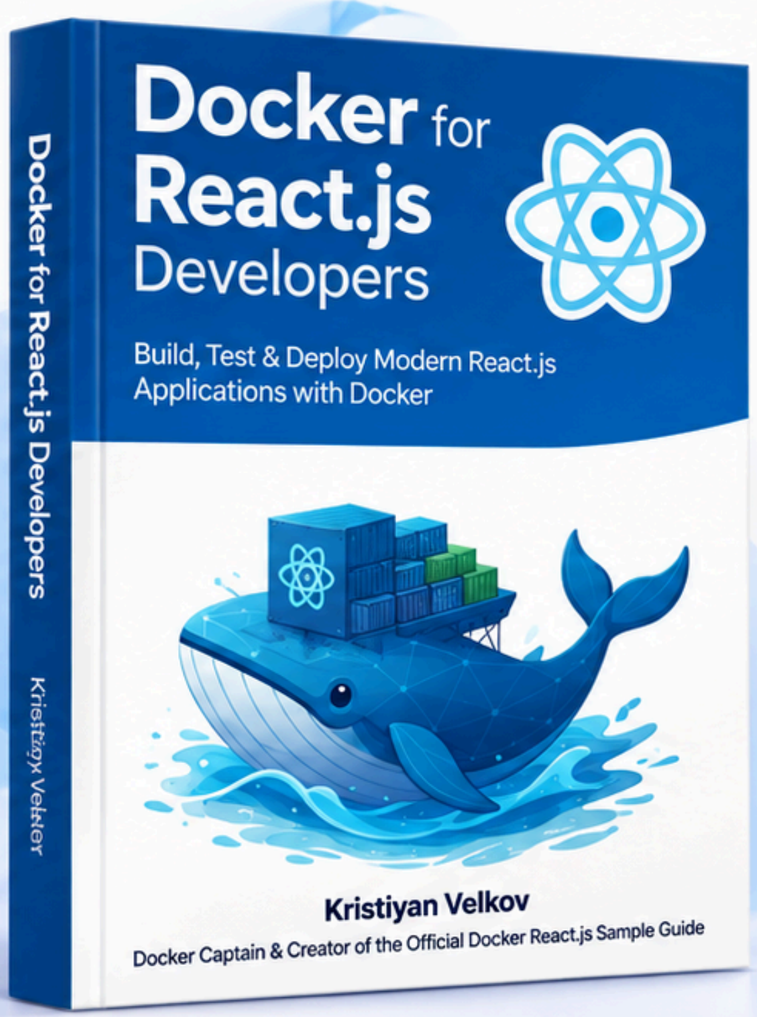
Don't trust anyone.
Verify everything.

- User input
- Client requests
- npm packages
- AI-written code
- LLM output

LIFE
DEMO
TIME!

HAPPY NEW YEAR
YOU HAVE BEEN HACKED

SUPPORT MY WORK AND BY MY NEW BOOK



Practical guide
Build, test and deploy React.js apps with Docker

Real examples
From development to production

Perfect for developers
Learn Docker in a practical, hands-on way

-30%

www.dockerfrontend.com

~~€45.99~~
€32.00

PROMO CODE
PROMO30

CONNECT WITH ME

Kristiyan Velkov

krisvelkov

Front-end Wolrd

Kristiyan Velkov

Front-End Advocate | Speaker | AI & DevOps Enthusiast | Docker Captain | ...

www.kristiyanvelkov.com